

CustomGate360

Documentación Técnica del Backend

Sistema de Gestion Aduanera con Motor de Perfilamiento de Riesgo

Stack tecnologico	Spring Boot 3 + PostgreSQL + Angular 20
Arquitectura	Controller -> Service -> Repository (MVC)
Motor de riesgo	SQL con 3 LEFT JOINS simultaneos
Base de datos	PostgreSQL 8 tablas ddl-auto=update
Puerto backend	localhost:8080
Puerto frontend	localhost:4200

1. Arquitectura General del Proyecto

El proyecto sigue el patron MVC dividido en dos aplicaciones independientes que se comunican por HTTP REST:

Capa	Tecnologia	Puerto	Responsabilidad
Frontend	Angular 20 (TypeScript)	4200	Interfaz de usuario, formularios, visualizacion
Backend	Spring Boot 3 (Java 21)	8080	API REST, logica de negocio, acceso a BD
Base de datos	PostgreSQL	5431	Persistencia, calculos SQL complejos

Estructura de paquetes del Backend

Todos los paquetes estan bajo **com.proyecto.backend**:

Subcarpeta	Descripcion
BackendApplication.java	Punto de entrada. Contiene el main() que arranca Spring Boot con @SpringBootApplication
config/	CorsConfig.java y DataInitializer.java — configuracion de CORS y datos semilla
controllers/	Reciben peticiones HTTP y delegan al service o repositorio. 6 controllers en total
dto/	Clases de transferencia de datos. NO son tablas BD. Solo empaquetan respuestas HTTP
models/	Entidades JPA: cada clase @Entity = una tabla en PostgreSQL. 8 entidades
repositories/	Interfaces Spring Data JPA. Generan SQL CRUD automaticamente. 8 repositorios
services/	Logica de negocio compleja: motor de riesgo y estadisticas. 2 services

2. Subcarpeta: config/

CorsConfig.java

El navegador bloquea peticiones de localhost:4200 (Angular) a localhost:8080 (Spring) porque son orígenes distintos. Esta clase con `@Configuration` registra un bean que autoriza esas peticiones:

Parametro	Valor configurado
Origen permitido	http://localhost:4200
Metodos HTTP	GET, POST, PUT, DELETE, OPTIONS
Headers	Todos (*)
Credenciales	Si (allowCredentials=true)
Rutas	Todas (**) — aplica a cada endpoint

DataInitializer.java

Implementa **CommandLineRunner**, lo que hace que Spring lo ejecute automáticamente al iniciar el servidor. Siembra los 3 catálogos del motor de riesgo:

Tabla	Estrategia	Cantidad	Descripción
catalogo_riesgo_pais	DELETE ALL + recarga completa	13 puertos	Ecuador (4), España (4), USA (5). Siempre actualizado al arrancar
importador_historial	Solo si tabla esta vacia	8 empresas	Incluye 2 infractoras (GlobalTrade Corp, Andina Logistics) con >2 infracciones
restricciones_arancelarias	Solo si tabla esta vacia	12 códigos	6 requieren permiso (armas, químicos, nucleares, oro) y 6 son estandar

3. Subcarpeta: models/ (Entidades JPA = Tablas PostgreSQL)

Cada clase @Entity es convertida por Hibernate en una tabla PostgreSQL. El parametro **ddl-auto=update** en application.properties hace que Hibernate cree o actualice las tablas automaticamente al iniciar. No es necesario escribir CREATE TABLE manualmente.

Clase Java	Tabla BD	Campos principales	Notas relevantes
OperacionAduanera	operaciones_aduaneras	id, numeroTracking (unique), tipoOperacion, estado, puertoOrigen, puertoDestino, fechaRegistro, idImportador, codigoArancelario, canalAforo, puntajeRiesgo	Tabla central. canalAforo y puntajeRiesgo son calculados por el motor de riesgo, no los ingresa el usuario
Usuario	usuario	id, username (unique), password, rol, cedula (unique, 10 digitos), nombreCompleto	Tiene validaciones @NotBlank y @Pattern. El rol por defecto es ADMIN
CatalogoRiesgoPais	catalogo_riesgo_pais	id, nombrePuertoOPais, nivelRiesgo (ALTO/MEDIO/BAJO), puntos (0/15/30), motivo	Primer vector del JOIN de riesgo. Se recarga completo en cada inicio del servidor
ImportadorHistorial	importador_historial	id, nombreEmpresa, rucEmpresa (unique), infraccionesPrevias, paisOrigen	Segundo vector del JOIN. Si infracciones > 2 suma 40 puntos al score de riesgo
RestriccionArancelaria	restricciones_arancelarias	id, codigoArancelario (unique), descripcion, requierePermiso (boolean), categoria	Tercer vector del JOIN. Si requierePermiso=true suma 30 puntos al score de riesgo
Pais	pais	id, nombre	Usa @Data de Lombok. Tabla de ubicacion geografica
Provincia	provincia	id, nombre, pais_id (FK)	@ManyToOne con Pais. Una provincia pertenece a un pais
Ciudad	ciudad	id, nombre, provincia_id (FK)	@ManyToOne con Provincia. Una ciudad pertenece a una provincia

4. Subcarpeta: repositories/

Todos extienden **JpaRepository<Entidad, Long>**. Spring Data JPA genera automaticamente el SQL para findAll(), findById(), save(), deleteById(), count(), existsById(). No se escribe SQL manualmente en esta capa.

Repositorio	Entidad	Metodos adicionales
OperacionAduaneraRepository	OperacionAduanera	findByNumeroTracking(String) -- Spring genera: SELECT * WHERE numero_tracking = ?
CatalogoRiesgoPaisRepository	CatalogoRiesgoPais	findByNombrePuertoOPaisIgnoreCase(String) -- busqueda sin distincion de mayusculas
ImportadorHistorialRepository	ImportadorHistorial	Solo metodos base de JpaRepository
RestriccionArancelariaRepository	RestriccionArancelaria	Solo metodos base de JpaRepository
UsuarioRepository	Usuario	findByUsername(String) -- usado en el login para verificar credenciales
PaisRepository	Pais	Solo metodos base
ProvinciaRepository	Provincia	findByPaisId(Long) -- filtra provincias segun el pais seleccionado
CiudadRepository	Ciudad	findByProvincialId(Long) -- filtra ciudades segun la provincia seleccionada

5. Subcarpeta: dto/ (Data Transfer Objects)

Los DTOs son clases que **solo transportan datos**. No tienen @Entity, no corresponden a tablas, y no tienen logica de negocio. Existen para estructurar las respuestas HTTP de forma clara y desacoplada del modelo de BD.

DetalleRiesgoDTO.java — El mas importante

Empaqueta el resultado completo del SQL con 3 LEFT JOINS. Tiene tres bloques de campos, uno por cada vector de riesgo:

Bloque	Campos	Tabla de origen del JOIN
Vector 1: Origen geografico	nivelRiesgoPais, puntosOrigen, motivoPais, origenEnCatalogo	catalogo_riesgo_pais
Vector 2: Historial importador	nombreImportador, infraccionesImportador, puntosImportador, importadorInfractor	importador_historial
Vector 3: Mercancia arancelaria	descripcionMercancia, categoriaMercancia, requierePermiso, puntosMercancia	restricciones_arancelarias
Resultado final	puntajeTotal, canalAforo, descripcionCanal	Calculado en Java segun los puntos obtenidos

OperacionConRiesgoResponse.java

Es el sobre de la respuesta del POST /api/operaciones. Contiene dos objetos anidados en un solo JSON:

Campo	Tipo Java	Descripcion
operacion	OperacionAduanera	La entidad ya guardada en BD con canalAforo y puntajeRiesgo asignados
detalleRiesgo	DetalleRiesgoDTO	El desglose de cada JOIN: que encontro y cuantos puntos sumo cada vector

EstadisticasDTO.java

Empaqueta los 4 resultados del dashboard para el endpoint GET /api/estadisticas:

Grupo de campos	Campos incluidos
Producto mas solicitado	productoCodigo, productoDescripcion, productoCategoria, productoTotalOperaciones
Puerto mas riesgoso	puertoNombre, puertoNivelRiesgo, puertoTotalOperaciones, puertoOperacionesRiesgo
Empresa mas infractora	empresaNombre, empresaInfracciones, empresaPaisOrigen
Totales generales	totalOperaciones, operacionesVerdes, operacionesAmarillos, operacionesRojas

6. Subcarpeta: services/

PerfilRiesgoService.java — El corazon del sistema

Motor de perfilamiento de riesgo aduanero. Usa **NamedParameterJdbcTemplate** en vez de JPA porque necesita SQL puro complejo con multiples JOINS que JPA no puede generar. Implementa el patron Semaforo de Aforos:

Canal	Rango de puntos	Accion automatica
VERDE	0 -- 30 pts	Desaduanizacion Automatica: estado pasa a DESADUANIZACION sin intervencion humana
AMARILLO	31 -- 70 pts	Aforo Documental: el Agente debe subir Factura, Certificado de Origen y Poliza de Seguro
ROJO	71 -- 100 pts	Aforo Fisico Intrusivo: el Inspector debe abrir el contenedor y llenar reporte de hallazgos

El service expone dos metodos publicos:

Metodo	Cuando se usa	Efecto sobre la entidad
evaluarYObtenerDetalle(op)	Al crear una nueva operacion (POST)	MUTA la entidad: asigna canalAforo, puntajeRiesgo, y si es VERDE cambia estado a DESADUANIZACION
analizarOperacionExistente(op)	Al consultar analisis de operacion ya guardada (GET /{id}/analisis)	NO muta la entidad. Solo devuelve el desglose para mostrar al usuario

EstadisticasService.java

Calcula 4 metricas globales con SQL directo. Tambien usa NamedParameterJdbcTemplate. Las consultas ejecutadas:

Metodo privado	Consulta SQL principal	Resultado
calcularTotalesPorCanal()	COUNT(*) + SUM(CASE WHEN canal_aforo=...) FROM operaciones_aduaneras	Cuantas ops son VERDE, AMARILLO y ROJO
calcularProductoMasSolicitado()	GROUP BY codigo_arancelario + LEFT JOIN restricciones_arancelarias ORDER BY COUNT DESC LIMIT 1	El codigo arancelario mas frecuente en operaciones
calcularPuertoMasRiesgo()	GROUP BY puerto_origen + LEFT JOIN catalogo_riesgo_pais, contando ops ROJO+AMARILLO	Puerto de origen con mas operaciones de riesgo
calcularEmpresaMasInfractora()	ORDER BY infracciones_previas DESC LIMIT 1	Empresa del importador_historial con mas infracciones

7. Subcarpeta: controllers/

Los controllers reciben las peticiones HTTP, llaman al service o repositorio, y devuelven la respuesta. Todos tienen `@CrossOrigin(origins='http://localhost:4200')`.

Controller	Base URL	Endpoints
OperacionAduaneraController	/api/operaciones	GET / POST / (+ motor riesgo) GET /tracking/{nro} PUT /{id}/estado GET /{id}/analisis GET /alerta-roja DELETE /{id}
CatalogoController	/api/catalogos	GET + POST /países-riesgo GET + POST /importadores GET + POST /arancelarios
UsuarioController	/api/admin	POST /login POST /registro (con validaciones) GET /listar DELETE /{id}
UbicacionController	/api/ubicaciones	GET /países GET /provincias/{paisId} GET /ciudades/{provincialId}
EstadisticasController	/api/estadisticas	GET / -- devuelve EstadisticasDTO con las 4 metricas
ApiController	/api	GET / -- health check basico del servidor

8. Relaciones entre Tablas SQL

8.1 Foreign Keys explícitas (anotaciones @ManyToOne en JPA)

Solo las tablas de ubicacion geografica tienen FK reales en la BD. El resto de relaciones son logicas (por valor de texto o ID):

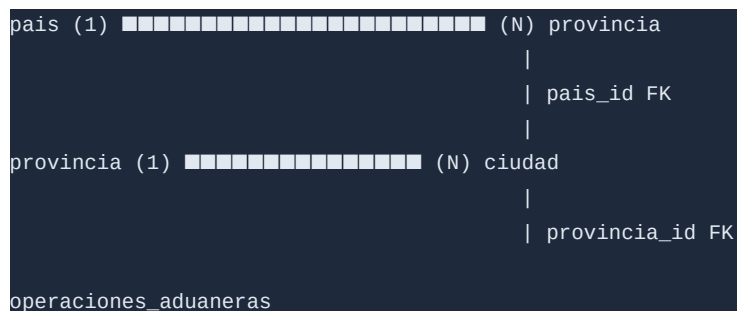
Tabla hija	Campo FK	Tabla padre	Campo referenciado	Tipo
provincia	pais_id	pais	id	MANY-TO-ONE: una provincia pertenece a un pais
ciudad	provincia_id	provincia	id	MANY-TO-ONE: una ciudad pertenece a una provincia

8.2 Relaciones logicas (usadas por el motor de riesgo en JOINS)

Estas relaciones **NO tienen FK** en la BD, pero el motor de riesgo las cruza via LEFT JOIN en tiempo de ejecucion:

Tabla	Campo de cruce	Tabla cruzada	Campo de cruce	Condicio n JOIN
operaciones_aduaneras	puerto_origen	catalogo_riesgo_pais	nombre_puerto_o_pais	LOWER(TRIM(crp.no mbre)) = L OWER(TR IM(:puerto Origen))
operaciones_aduaneras	id_importador	importador_historial	id	ih.id = :idl mportador (join por ID numerico)
operaciones_aduaneras	codigo_arancelario	restricciones_arancelarias	codigo_arancelario	LOWER(TRIM(ra.cod igo)) = LO WER(TRI M(:codigo Arancelari o))

8.3 Diagrama de relaciones



```
+-- puerto_origen ■ (LEFT JOIN texto) ■■> catalogo_riesgo_pais.nombre_puerto_o_pais
+-- id_importador ■ (LEFT JOIN ID) ■■> importador_historial.id
+-- codigo_arancelario (LEFT JOIN texto)■> restricciones_arancelarias.codigo_arancelario

usuario <■■■■■■■■■ tabla independiente (login / autenticacion)
```

9. La Consulta SQL Central: 3 LEFT JOINs Simultaneos

Esta es la consulta mas importante del sistema. Se ejecuta en PerfilRiesgoService cada vez que se crea o analiza una operacion. Todo el calculo del score ocurre **dentro de PostgreSQL**, no en Java:

```
SELECT
  -- Vector 1: catalogo_riesgo_pais
  CASE WHEN crp.id IS NOT NULL THEN true ELSE false END AS origen_en_catalogo,
  COALESCE(crp.nivel_riesgo, 'N/A') AS nivel_riesgo_pais,
  COALESCE(crp.puntos, 0) AS puntos_origen,
  COALESCE(crp.motivo, 'Puerto no catalogado') AS motivo_pais,

  -- Vector 2: importador_historial
  COALESCE(ih.nombre_empresa, 'Sin importador') AS nombre_importador,
  COALESCE(ih.infracciones_previas, 0) AS infracciones_importador,
  CASE WHEN COALESCE(ih.infracciones_previas, 0) > 2
    THEN 40 ELSE 0 END AS puntos_importador,

  -- Vector 3: restricciones_arancelarias
  COALESCE(ra.descripcion, 'Mercancia general') AS descripcion_mercancia,
  COALESCE(ra.requiere_permiso, false) AS requiere_permiso,
  CASE WHEN COALESCE(ra.requiere_permiso, false) = true
    THEN 30 ELSE 0 END AS puntos_mercancia,

  -- PUNTAJE TOTAL calculado en BD (no en Java)
  (COALESCE(crp.puntos, 0)
   + CASE WHEN COALESCE(ih.infracciones_previas,0) > 2 THEN 40 ELSE 0 END
   + CASE WHEN COALESCE(ra.requiere_permiso,false)=true THEN 30 ELSE 0 END
  ) AS puntaje_total

FROM (VALUES (1)) AS dummy(x)

LEFT JOIN catalogo_riesgo_pais crp
  ON LOWER(TRIM(crp.nombre_puerto_o_pais)) = LOWER(TRIM(:puertoOrigen))

LEFT JOIN importador_historial ih
  ON ih.id = :idImportador

LEFT JOIN restricciones_arancelarias ra
  ON LOWER(TRIM(ra.codigo_arancelario)) = LOWER(TRIM(:codigoArancelario))
```

Por que FROM (VALUES (1)) AS dummy(x)?

Es un truco SQL para hacer LEFT JOINs sin depender de una tabla real de origen. Si ninguno de los tres catalogos tiene coincidencia, la consulta igual devuelve una fila con los valores COALESCE por defecto (0 puntos, false). Garantiza que siempre devuelve exactamente UNA fila.

Calculo de puntos por vector

Vector	Condicion	Puntos	Max
Origen (puerto)	Puerto en catalogo con nivel ALTO	30	30

Origen (puerto)	Puerto en catalogo con nivel MEDIO	15	
Origen (puerto)	Puerto no catalogado o nivel BAJO	0	
Importador	infraccionesPrevias > 2	40	40
Importador	infraccionesPrevias <= 2 o no existe	0	
Mercancia	requierePermiso = true	30	30
Mercancia	Mercancia estandar o no existe en catalogo	0	
MAXIMO POSIBLE			100

10. Flujo Completo: Crear una Operacion Aduanera

Pa so	Actor	Accion
1	Frontend Angular	POST /api/operaciones con JSON de la operacion (puertoOrigen, idImportador, codigoArancelario, etc.)
2	OperacionAduaneraController	Recibe el @RequestBody. Fuerza estado = 'DOCUMENTACION'
3	PerfilRiesgoService	Ejecuta el SQL con 3 LEFT JOINS en PostgreSQL. Obtiene puntaje_total y detalles de cada vector
4	PerfilRiesgoService	Calcula canal: 0-30=VERDE, 31-70=AMARILLO, 71-100=ROJO. Si VERDE, cambia estado a 'DESADUANIZACION'
5	PerfilRiesgoService	Asigna canalAforo y puntajeRiesgo a la entidad operacion en memoria
6	OperacionAduaneraController	Persiste la operacion enriquecida via operacionRepository.save()
7	OperacionAduaneraController	Devuelve OperacionConRiesgoResponse: operacion guardada + DetalleRiesgoDTO
8	Frontend Angular	Muestra canal asignado, puntos de cada vector y descripcion del canal al usuario

Estados posibles de una operacion

Estado	Descripcion	Quien asigna
DOCUMENTACION	Estado inicial de toda operacion nueva	PerfilRiesgoService automaticamente al crear
DESADUANIZACION	Canal VERDE: pasa aqui directamente sin revision humana	PerfilRiesgoService automaticamente si score <= 30
LOGISTICA	Coordinacion logistica	PUT /{id}/estado -- asignacion manual
EN_TRANSITO	Carga en transito	PUT /{id}/estado -- asignacion manual
LLEGADA_PUERTO	Carga llego al puerto de destino	PUT /{id}/estado -- asignacion manual
CERRADO	Operacion finalizada	PUT /{id}/estado -- asignacion manual

11. Como Agregar una Tabla Nueva (Guia para la defensa)

Si el profesor pide agregar una nueva tabla, estos son los 4 pasos exactos siguiendo el patron del proyecto:

Paso	Archivo	Que hacer
1 -- Modelo	models/AgenteAduanero.java	@Entity + @Table(name='agentes_aduaneros') + campos con @Column + getters/setters. Hibernate crea la tabla automaticamente al arrancar gracias a ddl-auto=update
2 -- Repository	repositories/AgenteAduaneroRepository.java	public interface AgenteAduaneroRepository extends JpaRepository {} -- solo esa linea es suficiente para tener findAll, save, deleteById, etc.
3 -- Controller	controllers/CatalogoController.java o nuevo	Agregar @Autowired del repositorio + @GetMapping y @PostMapping con el nuevo endpoint
4 -- JOIN (opcional)	services/PerfilRiesgoService.java	Si la tabla debe influir en el score de riesgo, agregar LEFT JOIN adicional al SQL_ANALISIS_RIESGO y un nuevo CASE WHEN en el puntaje_total

Argumento tecnico: gracias a ddl-auto=update en application.properties, no hay que escribir ningun script SQL CREATE TABLE. Hibernate lee la nueva clase @Entity y crea la tabla automaticamente la proxima vez que arranca el servidor.

12. Resumen Completo de Endpoints REST

Metodo	Endpoint	Descripcion
GET	/api/operaciones	Lista todas las operaciones
POST	/api/operaciones	Crea operacion + ejecuta motor de riesgo completo (3 LEFT JOINS)
GET	/api/operaciones/tracking/{nro}	Busca operacion por numero de tracking
PUT	/api/operaciones/{id}/estado	Actualiza el estado de una operacion (avance manual)
GET	/api/operaciones/{id}/analisis	Re-ejecuta el JOIN de riesgo para operacion existente
GET	/api/operaciones/alerta-roja	Lista operaciones canal ROJO que no estan cerradas
DELETE	/api/operaciones/{id}	Elimina una operacion por ID
GET	/api/catalogos/paises-riesgo	Lista el catalogo de puertos y paises de riesgo
POST	/api/catalogos/paises-riesgo	Agrega un nuevo puerto/pais al catalogo
GET	/api/catalogos/importadores	Lista importadores con historial
POST	/api/catalogos/importadores	Agrega un importador nuevo
GET	/api/catalogos/arancelarios	Lista restricciones arancelarias
POST	/api/catalogos/arancelarios	Agrega una restriccion arancelaria
POST	/api/admin/login	Autenticacion de usuario (username + password plano)
POST	/api/admin/registro	Registra nuevo usuario admin con validaciones @NotBlank @Pattern
GET	/api/admin/listar	Lista todos los usuarios registrados
DELETE	/api/admin/{id}	Elimina un usuario por ID
GET	/api/ubicaciones/paises	Lista todos los paises
GET	/api/ubicaciones/provincias/{paisId}	Lista provincias de un pais
GET	/api/ubicaciones/ciudades/{provId}	Lista ciudades de una provincia
GET	/api/estadisticas	Devuelve las 4 metricas del dashboard (EstadisticasDTO)
GET	/api	Health check basico del servidor